

可靠的。我们看到了提供可靠的数据传送会遇到许多微妙的问题，但都可以通过精心地结合确认、定时器、重传以及序号机制来完成任务。

尽管本章中包含可靠数据传送，但是我们应该理解在链路层、网络层、运输层或应用层协议中都可以提供可靠数据传送。该协议栈中上面 4 层的任意一层都可以实现确认、定时器、重传以及序号，能够向其上层提供可靠数据传送。事实上，在过去数年中，工程师以及计算机科学家们已经独立地设计并实现了提供可靠数据传送的链路层、网络层、运输层以及应用层协议（虽然这些协议中的许多已经销声匿迹了）。

在 3.5 节中，我们详细地研究了 TCP 协议，它是因特网中面向连接和可靠的运输层协议。我们知道 TCP 是复杂的，它涉及连接管理、流量控制、往返时间估计以及可靠数据传送。事实上，TCP 比我们描述的要更为复杂，即我们有意地避而不谈在各种 TCP 实现版本中广泛实现的各种 TCP 补丁、修复和改进。然而，所有这些复杂性都对网络层应用隐藏了起来。如果某主机上的客户希望向另一台主机上的服务器可靠地发送数据，它只需要打开对该服务器的一个 TCP 套接字，然后将数据注入该套接字。客户-服务器应用程序则乐于对 TCP 的复杂性视而不见。

在 3.6 节中，我们从广泛的角度研究了拥塞控制，在 3.7 节中我们阐述了 TCP 是如何实现拥塞控制的。我们知道了拥塞控制对于网络良好运行是必不可少的。没有拥塞控制，网络很容易出现死锁，使得端到端之间很少或没有数据能被传输。在 3.7 节中我们学习了经典 TCP 实现的一种端到端拥塞控制机制，即当 TCP 连接的路径上判断不拥塞时，其传输速率就加性增；当出现丢包时，传输速率就乘性减。这种机制也致力于做到每一个通过拥塞链路的 TCP 连接能平等地共享该链路带宽。我们也研究了 TCP 拥塞控制的几个新的变种，它们使用基于时延的方法或来自网络的明确拥塞通告（而不是基于丢包的方法）来决定 TCP 发送速率，从而实现比经典 TCP 更快的决定速度。我们也更为深入地探讨了 TCP 连接建立和慢启动对时延的影响。我们观察到在许多重要场合，连接建立和慢启动会对端到端时延产生严重影响。我们再次强调，尽管 TCP 在这几年一直在发展，但它仍然是一个值得深入研究的领域，并且在未来的几年中还可能持续演化。为了圆满完成本章，3.8 节研究了实现运输层功能方面的新进展，如可靠数据传输、拥塞控制、连接建立等，这些方法在应用层都使用了 QUIC 协议。

在本章中我们对特定因特网运输协议的讨论集中在 UDP 和 TCP 上，它们是因特网运输层的两匹“骏马”。然而，对这两个协议的二十多年的经验已经使人们认识到，这两个协议都不是完美无缺的。研究人员因此在忙于研制其他的运输层协议，其中的几种现在已经成为 IETF 建议的标准。

在第 1 章中，我们讲到计算机网络能被划分成“网络边缘”和“网络核心”。网络边缘包含了在端系统中发生的所有事情。既然已经覆盖了应用层和运输层，我们关于网络边缘的讨论也就完成了。接下来是探寻网络核心的时候了！我们的旅程从下两章开始，第 5 章将学习网络层，第 6 章继续学习链路层。

课后习题和问题

复习题

3.1~3.3 节

R1. 假定网络层提供了下列服务。在源主机中的网络层接受最大长度 1200 字节和来自运输层的目的主机

地址的报文段。网络层则保证将该报文段交付给位于目的主机的运输层。假定在目的主机上能够运行许多网络应用进程。

- a. 设计尽可能简单的运输层协议, 该协议将使应用程序数据到达位于目的主机的所希望的进程。假定在目的主机中的操作系统已经为每个运行的应用进程分配了一个4字节的端口号。
 - b. 修改这个协议, 使它向目的进程提供一个“返回地址”。
 - c. 在你的协议中, 该运输层在计算机网络的核心中“必须做任何事”吗?
- R2. 考虑有一个星球, 每个人都属于某个六口之家, 每个家庭都住在自己的房子里, 每个房子都有唯一的地址, 并且某给定家庭中的每个人有一项独特的名字。假定该星球有一项从源家庭到目的家庭交付信件的邮政服务。该邮件服务要求: ①在一个信封中有一封信; ②在信封上清楚地写上目的家庭的地址 (并且没有别的东西)。假设每个家庭有一名家庭成员代表为家庭中的其他成员收集和分发信件。这些信没有必要提供任何有关信的接收者的指示。
- a. 根据复习题 R1 的解决方案, 描述家庭成员代表能够使用的协议, 以从发送家庭成员向接收家庭成员交付信件。
 - b. 在你的协议中, 该邮政服务必须打开信封并检查信件内容才能提供服务吗?
- R3. 考虑在主机 A 和主机 B 之间有一条 TCP 连接。假设从主机 A 传送到主机 B 的 TCP 报文段具有源端口号 x 和目的端口号 y 。对于从主机 B 传送到主机 A 的报文段, 源端口号和目的端口号分别是多少?
- R4. 描述应用程序开发者可能选择在 UDP 上运行应用程序而不是在 TCP 上运行的原因。
- R5. 在今天的因特网中, 为什么语音和图像流量常常是经过 TCP 而不是经 UDP 发送。(提示: 答案与 TCP 的拥塞控制机制没有关系。)
- R6. 当某应用程序运行在 UDP 上时, 该应用程序可能得到可靠数据传输吗? 如果能, 如何实现?
- R7. 假定在主机 C 上的一个进程有一个具有端口号 6789 的 UDP 套接字。假定主机 A 和主机 B 都用目的端口号 6789 向主机 C 发送一个 UDP 报文段。这两台主机的这些报文段在主机 C 都被描述为相同的套接字吗? 如果是这样的话, 在主机 C 的该进程将怎样知道源于两台不同主机的这两个报文段?
- R8. 假定在主机 C 的端口 80 上运行着一个 Web 服务器。假定这个 Web 服务器使用持续连接, 并且正在接收来自两台不同主机 A 和 B 的请求。被发送的所有请求都通过位于主机 C 的相同套接字吗? 如果它们通过不同的套接字传递, 这两个套接字都具有端口 80 吗? 讨论和解释之。

3.4 节

- R9. 在我们的 rdt 协议中, 为什么需要引入序号?
- R10. 在我们的 rdt 协议中, 为什么需要引入定时器?
- R11. 假定发送方和接收方之间的往返时延是固定的并且为发送方所知。假设分组能够丢失的话, 在协议 rdt3.0 中, 定时器仍是必需的吗? 试解释之。
- R12. 在配套网站上使用 Go-Back-N (回退 N 步) 动画。
- a. 让源发送 5 个分组, 在这 5 个分组的任何一个到达目的地之前暂停该动画。然后毁掉第一个分组并继续该动画。试描述发生的情况。
 - b. 重复该实验, 只是现在让第一个分组到达目的地并毁掉第一个确认。再次描述发生的情况。
 - c. 最后, 尝试发送 6 个分组。发生了什么情况?
- R13. 重复复习题 R12, 但是现在使用 Selective Repeat (选择重传) 动画。选择重传和回退 N 步有什么不同?

3.5 节

R14. 是非判断题:

- a. 主机 A 经过一条 TCP 连接向主机 B 发送一个大文件。假设主机 B 没有数据发往主机 A。因为主机 B 不能随数据捎带确认, 所以主机 B 将不向主机 A 发送确认。
- b. 在连接的整个过程中, TCP 的 `rwnd` 的长度绝不会变化。
- c. 假设主机 A 通过一条 TCP 连接向主机 B 发送一个大文件。主机 A 发送但未被确认的字节数不会超过接收缓存的大小。

- d. 假设主机 A 通过一条 TCP 连接向主机 B 发送一个大文件。如果对于这条连接的一个报文段的序号为 m ，则对于后继报文段的序号将必然是 $m+1$ 。
- e. TCP 报文段在它的首部中有一个 `rwnd` 字段。
- f. 假定在一条 TCP 连接中最后的 `SampleRTT` 等于 $1s$ ，那么对于该连接的 `TimeoutInterval` 的当前值必定大于等于 $1s$ 。
- g. 假设主机 A 通过一条 TCP 连接向主机 B 发送一个序号为 38 的 4 个字节的报文段。在这个相同的报文段中，确认号必定是 42。
- R15. 假设主机 A 通过一条 TCP 连接向主机 B 发送两个紧接着的 TCP 报文段。第一个报文段的序号为 90，第二个报文段序号为 110。
- a. 第一个报文段中有多少数据？
- b. 假设第一个报文段丢失而第二个报文段到达主机 B。那么在主机 B 发往主机 A 的确认报文中，确认号应该是多少？
- R16. 考虑在 3.5 节中讨论的 Telnet 的例子。在用户键入字符 C 数秒之后，用户又键入字符 R。那么在用户键入字符 R 之后，总共发送了多少个报文段，这些报文段中的序号和确认字段应该填入什么？

3.7 节

- R17. 假设两条 TCP 连接存在于一个带宽为 R 的瓶颈链路上。它们都要发送一个很大的文件（以相同方向经过瓶颈链路），并且两者是同时开始发送文件。那么 TCP 将为每条连接分配什么样的传输速率？
- R18. 是非判断题。考虑 TCP 的拥塞控制。当发送方定时器超时，其 `ssthresh` 的值将被设置为原来值的一半。
- R19. 在 3.7 节的“TCP 分岔”讨论中，对于 TCP 分岔的响应时间，断言大约是 $4RTT_{FE} + RTT_{RE} + \text{处理时间}$ 。评价该断言。

习题

- P1. 假设客户 A 向服务器 S 发起一个 Telnet 会话。与此同时，客户 B 也向服务器 S 发起一个 Telnet 会话。给出下面报文段的源端口号和目的端口号：
- a. 从 A 向 S 发送的报文段。
- b. 从 B 向 S 发送的报文段。
- c. 从 S 向 A 发送的报文段。
- d. 从 S 向 B 发送的报文段。
- e. 如果 A 和 B 是不同的主机，那么从 A 向 S 发送的报文段的源端口号是否可能与从 B 向 S 发送的报文段的源端口号相同？
- f. 如果它们是同一台主机，情况会怎么样？
- P2. 考虑图 3-5。从服务器返回客户进程的报文流中的源端口号和目的端口号是多少？在承载运输层报文段的网络层数据报中，IP 地址是多少？
- P3. UDP 和 TCP 使用反码来计算检验和。假设你有下面 3 个 8 比特字节：01010011，01100110，01110100。这些 8 比特字节和的反码是多少？（注意到尽管 UDP 和 TCP 使用 16 比特的字来计算检验和，但对于这个问题，你应该考虑 8 比特和。）写出所有工作过程。UDP 为什么要用该和的反码，即为什么不直接使用该和呢？使用该反码方案，接收方如何检测出差错？1 比特的差错将可能检测不出来吗？2 比特的差错呢？
- P4. a. 假定你有下列 2 个字节：01011100 和 01100101。这 2 个字节之和的反码是什么？
- b. 假定你有下列 2 个字节：11011010 和 01100101。这 2 个字节之和的反码是什么？
- c. 对于 (a) 中的字节，给出一个例子，使得这 2 个字节中的每一个都在一个比特反转时，其反码不会改变。
- P5. 假定某 UDP 接收方对接收到的 UDP 报文段计算因特网检验和，并发现它与承载在检验和字段中的值

相匹配。该接收方能够绝对确信没有出现过比特差错吗？试解释之。

- P6. 考虑我们改正协议 rdt2.1 的动机。试说明如图 3-60 所示的接收方与如图 3-11 所示的发送方运行时，接收方可能会引起发送方和接收方进入死锁状态，即双方都在等待不可能发生的事件。

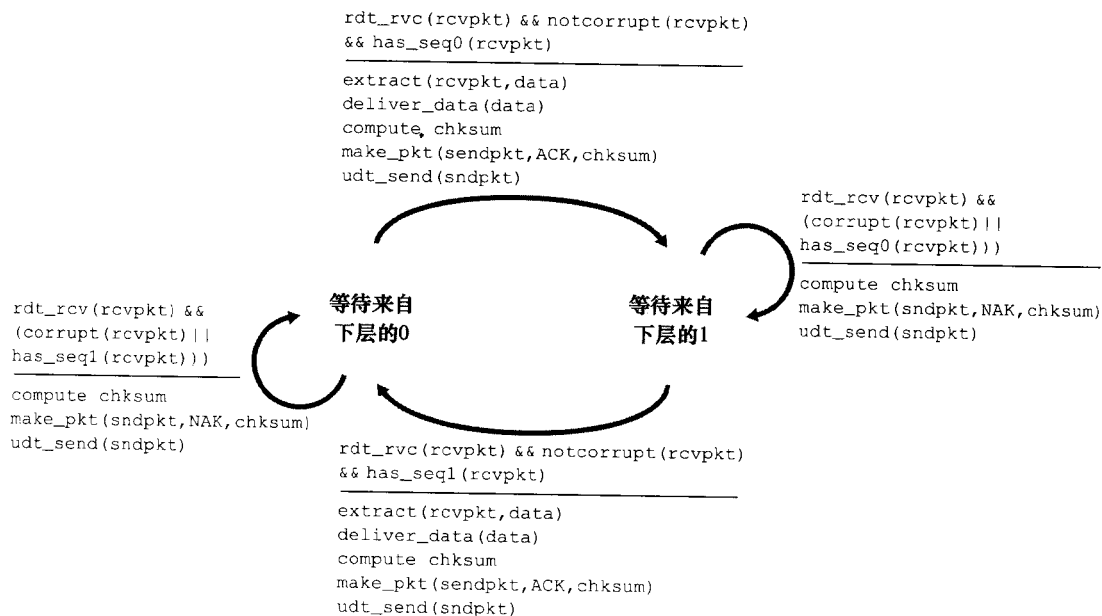


图 3-60 协议 rdt2.1 的一个不正确的接收方

- P7. 在 rdt3.0 协议中，从接收方向发送方流动的 ACK 分组没有序号（尽管它们具有 ACK 字段，该字段包括了它们正在确认的分组的序号）。为什么这些 ACK 分组不需要序号呢？
- P8. 画出协议 rdt3.0 中接收方的 FSM。
- P9. 当数据分组和确认分组发生篡改时，给出 rdt3.0 协议运行的轨迹。你画的轨迹应当类似于图 3-16 中所用的图。
- P10. 考虑一个能够丢失分组但其最大时延已知的信道。修改协议 rdt2.1，以包括发送方超时和重传机制。非正式地论证：为什么你的协议能够通过该信道正确通信？
- P11. 考虑在图 3-14 中的 rdt2.2 接收方，在状态“等待来自下层的 0”和状态“等待来自下层的 1”中的自转换（即从某状态转换回自身）中生成一个新分组：`sndpkt = make_pkt(ACK, 1, checksum)`和 `sndpkt = make_pkt(ACK, 0, checksum)`。如果这个操作从状态“等待来自下层的 1”中的自转换中删除，该协议将正确工作吗？评估你的答案。在状态“等待来自下层的 0”中的自转换中删除这个事件将会怎样？（提示：在后一种情况下，考虑如果第一个发送方到接收方的分组损坏的话，将会发生什么情况？）
- P12. rdt3.0 协议的发送方直接忽略（即不采取任何操作）接收到的所有出现差错和确认分组的确认号（acknum）字段中的值有差错的分组。假设在这种情况下，rdt3.0 只是重传当前的数据分组，该协议是否还能正常运行？（提示：考虑在下列情况下会发生什么情况：仅有一个比特差错时；报文没有丢失但能出现定时器过早超时。考虑到当 n 趋于无穷时，第 n 个分组将被发送多少次。）
- P13. 考虑 rdt3.0 协议。如果发送方和接收方的网络连接能够对报文重排序（即在发送方和接收方之间的媒介上传播的两个报文段能重新排序），那么比特交替协议将不能正确工作（确信你清楚地理解这时它不能正确工作的原因），试画图说明之。画图时把发送方放在左边，接收方放在右边，使时间轴朝下，标出交换的数据报文（D）和确认报文（A）。要标明与任何数据和确认报文段相关的序号。
- P14. 考虑一种仅使用否定确认的可靠数据传输协议。假定发送方只是偶尔发送数据。只用 NAK 的协议

是否会比使用 ACK 的协议更好？为什么？现在我们假设发送方要发送大量的数据，并且该端到端连接很少丢包。在第二种情况下，只用 NAK 的协议是否会比使用 ACK 的协议更好？为什么？

- P15. 考虑显示在图 3-17 中的网络跨越国家的例子。窗口长度设置成多少时，才能使该信道的利用率超过 90%？假设分组的长度为 1500 字节（包括首部字段和数据）。
- P16. 假设某应用使用 rdt3.0 作为其运输层协议。因为停等协议具有非常低的信道利用率（显示在网络跨越国家的例子中），该应用程序的设计者让接收方持续回送许多（大于 2）交替的 ACK 0 和 ACK 1，即使对应的数据未到达接收方。这个应用程序设计将能增加信道利用率吗？为什么？该方法存在某种潜在的问题吗？试解释之。
- P17. 考虑两个网络实体 A 和 B，它们由一条完善的双向信道所连接（即任何发送的报文将正确地收到；信道将不会损坏、丢失或重排序分组）。A 和 B 将以交互的方式彼此交付报文：首先，A 必须向 B 交付一个报文，B 然后必须向 A 交付一个报文，接下来 A 必须向 B 交付一个报文，等等。如果一个实体处于它不试图向另一侧交付报文的状态，将存在一个来自上层的类似于 `rdt_send(data)` 调用的事件，它试图向下传送数据以向另一侧传输，来自上层的该调用能够直接忽略对于 `rdt_unable_to_send(data)` 调用，这通知较高层当前不能够发送数据。（注意：做出这种简化的假设，使你不必担心缓存数据。）

为该协议画出 FSM 说明（一个 FSM 用于 A，一个 FSM 用于 B）。注意，不必担心这里的可靠性机制，该问题的要点在于创建反映这两个实体的同步行为的 FSM 说明。应当使用与图 3-9 中协议 rdt1.0 有相同含义的下列事件和操作：`rdt_send(data)`，`packet = make_pkt(data)`，`udt_send(data)`，`rdt_rcv(packet)`，`extract(packet, data)`，`deliver_data(data)`。保证你的协议反映了 A 和 B 之间发送的严格交替。还要保证在你的 FSM 描述中指出 A 和 B 的初始状态。

- P18. 在 3.4.4 节我们学习的一般性 SR 协议中，只要报文可用（如果报文在窗口中），发送方就会不等待确认而传输报文。假设现在我们要求一个 SR 协议，一次发出一对报文，而且只有在知道第一对报文中的一个报文都正确到达后才发送第二对报文。

假设该信道中可能会丢失报文，但报文不会发生损坏和失序。试为报文的单向可靠传输而设计一个差错控制协议。画出发送方和接收方的 FSM 描述。描述在发送方和接收方之间两个方向发送的报文格式。如果你使用了不同于 3.4 节 [例如 `udt_send()`、`start_timer()`、`rdt_rcv()` 等] 中的任何其他过程调用，详细地阐述这些操作。举例说明（用发送方和接收方的时序踪迹图）你的协议是如何恢复报文丢失的。

- P19. 考虑一种情况，主机 A 想同时向主机 B 和主机 C 发送分组。A 与 B 和 C 是经过广播信道连接的，即由 A 发送的分组通过该信道传送到 B 和 C。假设连接 A、B 和 C 的这个广播信道具有独立的报文丢失和损坏特性（例如，从 A 发出的报文可能被 B 正确接收，但没有被 C 正确接收）。设计一个类似于停等协议的差错控制协议，用于从 A 可靠地传输分组到 B 和 C。该协议使得 A 直到得知 B 和 C 已经正确接收到当前报文，才获取上层交付的新数据。给出 A 和 C 的 FSM 描述。（提示：B 的 FSM 大体上应当与 C 的相同。）同时，给出所使用的报文格式的描述。
- P20. 考虑一种主机 A 和主机 B 要向主机 C 发送报文的情况。主机 A 和 C 通过一条报文能够丢失和损坏（但不重排序）的信道相连接。主机 B 和 C 由另一条（与连接 A 和 C 的信道独立）具有相同性质的信道连接。在主机 C 上的运输层，在向上层交付来自主机 A 和 B 的报文时应当交替进行（即它应当首先交付来自 A 的分组中的数据，然后是来自 B 的分组中的数据，等等）。设计一个类似于停等协议的差错控制协议，以可靠地向 C 传输来自 A 和 B 的分组，同时以前面描述的方式在 C 处交替地交付。给出 A 和 C 的 FSM 描述。（提示：B 的 FSM 大体上应当与 A 的相同。）同时，给出所使用的报文格式的描述。
- P21. 假定我们有两个网络实体 A 和 B。B 有一些数据报文要通过下列规则传给 A。当 A 从其上层得到一个请求，就从 B 获取下一个数据（D）报文。A 必须通过 A-B 信道向 B 发送一个请求（R）报文。仅当 B 收到一个 R 报文后，它才会通过 B-A 信道向 A 发送一个数据（D）报文。A 应当准确地将每份 D 报文的副本交付给上层。R 报文可能会在 A-B 信道中丢失（但不会损坏）；D 报文一旦发出总

是能够正确交付。两个信道的时延未知且是变化的。

设计一个协议（给出 FSM 描述），它能够综合适当的机制，以补偿会丢包的 A-B 信道，并且实现在 A 实体中向上层传递报文。只采用绝对必要的机制。

- P22. 考虑一个 GBN 协议，其发送方窗口为 4，序号范围为 1024。假设在时刻 t ，接收方期待的下一个有序分组的序号是 k 。假设媒介不会对报文重新排序。回答以下问题：
- 在 t 时刻，发送方窗口内的报文序号可能是多少？论证你的回答。
 - 在 t 时刻，在当前传播回发送方的所有可能报文中，ACK 字段的所有可能值是多少？论证你的回答。
- P23. 考虑 GBN 协议和 SR 协议。假设序号空间的长度为 k ，那么为了避免出现图 3-27 中的问题，对于这两种协议中的每一种，允许的发送方窗口最大为多少？
- P24. 对下面的问题判断是非，并简要地证实你的回答：
- 对于 SR 协议，发送方可能会收到落在其当前窗口之外的分组的 ACK。
 - 对于 GBN 协议，发送方可能会收到落在其当前窗口之外的分组的 ACK。
 - 当发送方和接收方窗口长度都为 1 时，比特交替协议与 SR 协议相同。
 - 当发送方和接收方窗口长度都为 1 时，比特交替协议与 GBN 协议相同。
- P25. 我们曾经说过，应用程序可能选择 UDP 作为运输协议，因为 UDP 提供了（比 TCP）更好的应用层控制，以决定在报文段中发送什么数据和发送时机。
- 应用程序为什么对在报文段中发送什么数据有更多的控制？
 - 应用程序为什么对何时发送报文段有更多的控制？
- P26. 考虑从主机 A 向主机 B 传输 L 字节的大文件，假设 MSS 为 536 字节。
- 为了使得 TCP 序号不至于用完， L 的最大值是多少？前面讲过 TCP 的序号字段为 4 字节。
 - 对于你在（a）中得到的 L ，传输此文件要用多长时间？假定运输层、网络层和数据链路层首部总共为 66 字节，并加在每个报文段上，然后经 155Mbps 链路发送得到的分组。忽略流量控制和拥塞控制，使主机 A 能够一个接一个和连续不断地发送这些报文段。
- P27. 主机 A 和 B 经一条 TCP 连接通信，并且主机 B 已经收到了来自 A 的最长为 126 字节的所有字节。假定主机 A 随后向主机 B 发送两个紧接着的报文段。第一个和第二个报文段分别包含了 80 字节和 40 字节的数据。在第一个报文段中，序号是 127，源端口号是 302，目的地端口号是 80。无论何时主机 B 接收到来自主机 A 的报文段，它都会发送确认。
- 在从主机 A 发往 B 的第二个报文段中，序号、源端口号和目的端口号各是什么？
 - 如果第一个报文段在第二个报文段之前到达，在第一个到达报文段的确认中，确认号、源端口号和目的端口号各是什么？
 - 如果第二个报文段在第一个报文段之前到达，在第一个到达报文段的确认中，确认号是什么？
 - 假定由 A 发送的两个报文段按序到达 B。第一个确认丢失了而第二个确认在第一个超时间隔之后到达。画出时序图，显示这些报文段和发送的所有其他报文段和确认。（假设没有其他分组丢失。）对于图上每个报文段，标出序号和数据的字节数量；对于你增加的每个应答，标出确认号。
- P28. 主机 A 和 B 直接经一条 100Mbps 链路连接。在这两台主机之间有一条 TCP 连接。主机 A 经这条连接向主机 B 发送一个大文件。主机 A 能够向它的 TCP 套接字以高达 120Mbps 的速率发送应用数据，而主机 B 能够以最大 50Mbps 的速率从它的 TCP 接收缓存中读出数据。描述 TCP 流量控制的影响。
- P29. 在 3.5.6 节中讨论了 SYN cookie。
- 服务器在 SYNACK 中使用一个特殊的初始序号，这为什么是必要的？
 - 假定某攻击者得知一台目标主机使用了 SYN cookie。该攻击者能够通过直接向目标发送一个 ACK 分组创建半开或全开连接吗？为什么？
 - 假设某攻击者收集了由服务器发送的大量初始序号。该攻击者通过发送具有初始序号的 ACK，能够引起服务器产生许多全开连接吗？为什么？
- P30. 考虑在 3.6.1 节中显示在第二种情况下的网络。假设发送主机 A 和 B 具有某些固定的超时值。
- 证明增加路由器有限缓存的长度可能减小吞吐量（ λ_{out} ）。

- b. 现在假设两台主机基于路由器的缓存时延, 动态地调整它们的超时值 (像 TCP 所做的那样)。增加缓存长度将有助于增加吞吐量吗? 为什么?
- P31. 假设测量的 5 个 SampleRTT 值 (参见 3.5.3 节) 是 106ms、120ms、140ms、90ms 和 115ms。在获得了每个 SampleRTT 值后计算 EstimatedRTT, 使用 $\alpha=0.125$ 并且假设在刚获得前 5 个样本之后 EstimatedRTT 的值为 100ms。在获得每个样本之后, 也计算 DevRTT, 假设 $\beta=0.25$, 并且假设在刚获得前 5 个样本之后 DevRTT 的值为 5ms。最后, 在获得这些样本之后计算 TCP TimeoutInterval。
- P32. 考虑 TCP 估计 RTT 的过程。假设 $\alpha=0.1$, 令 SampleRTT_{T_1} 设置为最新样本 RTT, 令 SampleRTT_{T_2} 设置为下一个最新样本 RTT, 等等。
- a. 对于一个给定的 TCP 连接, 假定 4 个确认报文相继到达, 带有 4 个对应的 RTT 值: SampleRTT_{T_4} 、 SampleRTT_{T_3} 、 SampleRTT_{T_2} 和 SampleRTT_{T_1} 。根据这 4 个样本 RTT 表示 EstimatedRTT。
- b. 将你得到的公式一般化到 n 个 RTT 样本的情况。
- c. 对于在 (b) 中得到的公式, 令 n 趋于无穷。试说明为什么这个平均过程被称为指数移动平均。
- P33. 在 3.5.3 节中, 我们讨论了 TCP 的往返时间的估计。TCP 避免测量重传报文段的 SampleRTT, 对此你有什么看法?
- P34. 3.5.4 节中的变量 SendBase 和 3.5.5 节中的变量 LastByteRcvd 之间有什么关系?
- P35. 3.5.5 节中的变量 LastByteRcvd 和 3.5.4 节中的变量 γ 之间有什么关系?
- P36. 在 3.5.4 节中, 我们看到 TCP 直到收到 3 个冗余 ACK 才执行快速重传。你对 TCP 设计者没有选择在收到对报文段的第一个冗余 ACK 后就快速重传有何看法?
- P37. 比较 GBN、SR 和 TCP (无延时的 ACK)。假设对所有 3 个协议的超时值足够长, 使得 5 个连续的数据报文段及其对应的 ACK 能够分别由接收主机 (主机 B) 和发送主机 (主机 A) 收到 (如果在信道中无丢失)。假设主机 A 向主机 B 发送 5 个数据报文段, 并且第二个报文段 (从 A 发送) 丢失。最后, 所有 5 个数据报文段已经被主机 B 正确接收。
- a. 主机 A 总共发送了多少报文段和主机 B 总共发送了多少 ACK? 它们的序号是什么? 对所有 3 个协议回答这个问题。
- b. 如果对所有 3 个协议超时值比 5RTT 长得多, 则哪个协议在最短的时间间隔中成功地交付所有 5 个数据报文段?
- P38. 在图 3-52 的 TCP 描述中, 阈值 ssthresh 的值在几个地方被设置为 $\text{ssthresh}=\text{cwnd}/2$, 并且当出现一个丢包事件时, ssthresh 的值被设置为窗口长度的一半。当出现丢包事件时, 发送方发送的速率必须大约等于 cwnd 个报文段每 RTT 吗? 解释你的答案。如果你的回答是没有, 你能提出一种不同的方式来进行 ssthresh 设置吗?
- P39. 考虑图 3-46b。如果 λ'_{in} 增加超过了 $R/2$, λ_{out} 能够增加超过 $R/3$ 吗? 试解释之。现在考虑图 3-46c。假定一个分组从路由器到接收方平均转发两次, 如果 λ'_{in} 增加超过 $R/2$, λ_{out} 能够增加超过 $R/4$ 吗? 试解释之。
- P40. 考虑图 3-61。假设 TCP Reno 是一个经历如上所示行为的协议, 回答下列问题。在各种情况中, 简要地论证你的回答。
- a. 指出 TCP 慢启动运行时的时间间隔。
- b. 指出 TCP 拥塞避免运行时的时间间隔。
- c. 在第 16 个传输轮回之后, 报文段的丢失是根据 3 个冗余 ACK 还是根据超时检测出来的?
- d. 在第 22 个传输轮回之后, 报文段的丢失是根据 3 个冗余 ACK 还是根据超时检测出来的?
- e. 在第 1 个传输轮回里, ssthresh 的初始值设置为多少?

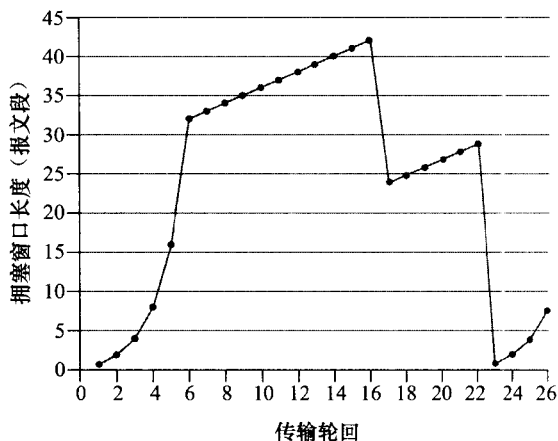


图 3-61 TCP 窗口长度作为时间的函数

- f. 在第 18 个传输轮回里, ssthresh 的值设置为多少?
- g. 在第 24 个传输轮回里, ssthresh 的值设置为多少?
- h. 在哪个传输轮回内发送第 70 个报文段?
- i. 假定在第 26 个传输轮回后, 通过收到 3 个冗余 ACK 检测出有分组丢失, 拥塞的窗口长度和 ssthresh 的值应当是多少?
- j. 假定使用 TCP Tahoe (而不是 TCP Reno), 并假定在第 16 个传输轮回收到 3 个冗余 ACK。在第 19 个传输轮回, ssthresh 和拥塞窗口长度是什么?
- k. 再次假设使用 TCP Tahoe, 在第 22 个传输轮回有一个超时事件。从第 17 个传输轮回回到第 22 个传输轮回 (包括这两个传输轮回), 一共发送了多少分组?
- P41. 参考图 3-55, 该图描述了 TCP 的 AIMD 算法的收敛特性。假设 TCP 不采用乘性减, 而是采用按某一常量减小窗口。所得的 AIAD 算法将收敛于一种平等共享算法吗? 使用类似于图 3-55 中的图来证实你的结论。
- P42. 在 3.5.4 节中, 我们讨论了在发生超时事件后将超时时间隔加倍。为什么除了这种加倍超时时间隔机制外, TCP 还需要基于窗口的拥塞控制机制 (如在 3.7 节中学习的那种机制) 呢?
- P43. 主机 A 通过一条 TCP 连接向主机 B 发送一个很大的文件。在这条连接上, 不会出现任何分组丢失和定时器超时。主机 A 与因特网连接链路的传输速率表示为 R bps。假设主机 A 上的进程能够以 S bps 的速率向 TCP 套接字发送数据, 其中 $S = 10R$ 。进一步假设 TCP 的接收缓存足够大, 能够容纳整个文件, 并且发送缓存只能容纳这个文件的百分之一。如何防止主机 A 上的进程连续地向 TCP 套接字以速率 S bps 传送数据呢? 还是用 TCP 流量控制呢? 还是用 TCP 拥塞控制? 或者用其他措施? 阐述其理由。
- P44. 考虑从一台主机经一条没有丢包的 TCP 连接向另一台主机发送一个大文件。
- 假定 TCP 使用不具有慢启动的 AIMD 进行拥塞控制。假设每当收到一批 ACK 时, cwnd 增加 1 个 MSS, 并且假设往返时间大约恒定, cwnd 从 6MSS 增加到 12MSS 要花费多长时间 (假设没有丢包事件)?
 - 对于该连接, 到时间 $= 6RTT$, 其平均吞吐量是多少 (根据 MSS 和 RTT)?
- P45. 考虑图 3-54, 假设在 t_3 时刻, 即下一个拥塞丢包发生时, 发送速率下降为 $0.75W_{\max}$ (当然, 不为 TCP 发送方所知)。请分别给出 TCP Reno 和 TCP CUBIC 在之后两轮的变化情况。(提示: TCP Reno 和 TCP CUBIC 对拥塞丢包做出反应的时间可能不再相同。)
- P46. 再次考虑图 3-54, 假设在 t_3 时刻, 即下一个拥塞丢包发生时, 发送速率上升为 $1.5W_{\max}$ 。请分别给出 TCP Reno 和 TCP CUBIC 在之后两轮的变化情况。(提示同上一题。)
- P47. 回想 TCP 吞吐量的宏观描述。在连接速率从 $W/(2RTT)$ 变化到 W/RTT 的周期内, 只丢失了一个分组 (在该周期的结束)。
- 证明其丢包率 (分组丢失的比率) 等于:

$$L = \text{丢包率} = \frac{1}{\frac{3}{8}W^2 + \frac{3}{4}W}$$

- 如果一条连接的丢包率为 L , 使用上面的结果, 则它的平均速率近似由下式给出:

$$\text{平均速率} \approx \frac{1.22\text{MSS}}{RTT\sqrt{L}}$$

- P48. 考虑仅有一条单一的 TCP (Reno) 连接使用一条 10Mbps 链路, 且该链路没有缓存任何数据。假设这条链路是发送主机和接收主机之间的唯一拥塞链路。假定某 TCP 发送方向接收方有一个大文件要发送, 而接收方的接收缓存比拥塞窗口要大得多。我们也做下列假设: 每个 TCP 报文段长度为 1500 字节; 该连接的双向传播时延是 150ms; 并且该 TCP 连接总是处于拥塞避免阶段, 即忽略了慢启动。
- 这条 TCP 连接能够取得的最大窗口长度 (以报文段计) 是多少?
 - 这条 TCP 连接的平均窗口长度 (以报文段计) 和平均吞吐量 (以 bps 计) 是多少?
 - 这条 TCP 连接在从丢包恢复后, 再次到达其最大窗口要经历多长时间?

- P49. 考虑在前面习题中所描述的场景。假设 10Mbps 链路能够缓存有限个报文段。试论证为了使该链路总是忙于发送数据，我们将要选择缓存长度，使得其至少为发送方和接收方之间链路速率 C 与双向传播时延之积。
- P50. 重复习题 46，但用一条 10Gbps 链路代替 10Mbps 链路。注意到在对 c 部分的答案中，应当认识到在从丢包恢复后，拥塞窗口长度到达最大窗口长度将需要很长时间。给出解决该问题的基本思路。
- P51. 令 T （用 RTT 度量）表示一条 TCP 连接将拥塞窗口从 $W/2$ 增加到 W 所需的时间间隔，其中 W 是最大的拥塞窗口长度。论证 T 是 TCP 平均吞吐量的函数。
- P52. 考虑一种简化的 TCP 的 AIMD 算法，其中拥塞窗口长度用报文段的数量来度量，而不是用字节度量。在加性增中，每个 RTT 拥塞窗口长度增加一个报文段。在乘性减中，拥塞窗口长度减小一半（如果结果不是一个整数，向下取整到最近的整数）。假设两条 TCP 连接 C1 和 C2，它们共享一条速率为每秒 30 个报文段的单一拥塞链路。假设 C1 和 C2 均处于拥塞避免阶段。连接 C1 的 RTT 是 50ms，连接 C2 的 RTT 是 100ms。假设当链路中的数据速率超过了链路的速率时，所有 TCP 连接经受数据报文段丢失。
- 如果在时刻 t_0 ，C1 和 C2 具有 10 个报文段的拥塞窗口，在 1000ms 后它们的拥塞窗口为多长？
 - 经长时间运行，这两条连接将取得共享该拥塞链路的相同的带宽吗？
- P53. 考虑在前面习题中描述的网络。现在假设两条 TCP 连接 C1 和 C2，它们具有相同的 100ms RTT。假设在时刻 t_0 ，C1 的拥塞窗口长度为 15 个报文段，而 C2 的拥塞窗口长度是 10 个报文段。
- 在 2200ms 后，它们的拥塞窗口长度为多长？
 - 经长时间运行，这两条连接将取得共享该拥塞链路的相同的带宽吗？
 - 如果这两条连接在相同时间达到它们的最大窗口长度，并在相同时间达到它们的最小窗口长度，我们说这两条连接是同步的。经长时间运行，这两条连接将最终变得同步吗？如果是，它们的最大窗口长度是多少？
 - 这种同步将有助于改善共享链路的利用率吗？为什么？给出打破这种同步的某种思路。
- P54. 考虑修改 TCP 的拥塞控制算法。不使用加性增，使用乘性增。无论何时某 TCP 收到一个合法的 ACK，就将其窗口长度增加一个小正数 a ($0 < a < 1$)。求出丢包率 L 和最大拥塞窗口 W 之间的函数关系。论证：对于这种修正的 TCP，无论 TCP 的平均吞吐量如何，一条 TCP 连接将其拥塞窗口长度从 $W/2$ 增加到 W ，总是需要相同的时间。
- P55. 在 3.7 节对 TCP 未来的讨论中，我们注意到为了达到 10Gbps 的吞吐量，TCP 仅能容忍 2×10^{-10} 的报文段丢失率（或等价于每 5 000 000 000 个报文段有一个丢包事件）。给出针对 3.7 节中给定的 RTT 和 MSS 值的对 2×10^{-10} 值的推导。如果 TCP 需要支持一条 100Gbps 的连接，所能容忍的丢包率是多少？
- P56. 在 3.7 节中对 TCP 拥塞控制的讨论中，我们隐含地假定 TCP 发送方总是有数据要发送。现在考虑下列情况，某 TCP 发送方发送大量数据，然后在 t_1 时刻变得空闲（因为它没有更多的数据要发送）。TCP 在相对长的时间内保持空闲，然后在 t_2 时刻要发送更多的数据。当 TCP 在 t_2 开始发送数据时，让它使用在 t_1 时刻的 cwnd 和 ssthresh 值，将有什么样的优点和缺点？你建议使用什么样的方法？为什么？
- P57. 在这个习题中我们研究是否 UDP 或 TCP 提供了某种程度的端点鉴别。
- 考虑一台服务器接收到在一个 UDP 分组中的请求并对该请求进行响应（例如，如由 DNS 服务器所做的那样）。如果一个具有 IP 地址 X 的客户用地址 Y 进行哄骗的话，服务器将向何处发送它的响应？
 - 假定一台服务器接收到具有 IP 源地址 Y 的一个 SYN，在用 SYNACK 响应之后，接收一个具有 IP 源地址 Y 和正确确认号的 ACK。假设该服务器选择了一个随机初始序号并且没有“中间人”，该服务器能够确定该客户的确位于 Y 吗？（并且不在某个其他哄骗为 Y 的地址 X。）
- P58. 在这个习题中，我们考虑由 TCP 慢启动阶段引入的时延。考虑一个客户和一个 Web 服务器直接连接到速率 R 的一条链路。假定该客户要取回一个对象，其长度正好等于 $15S$ ，其中 S 是最大段长度（MSS）。客户和服务器之间的往返时间表示为 RTT（假设为常数）。忽略协议首部，确定在下列情况下取回该对象的时间（包括 TCP 连接创建）：

- a. $4S/R > S/R + RTT > 2S/R$
- b. $S/R + RTT > 4S/R$
- c. $S/R > RTT$



编程作业

实现一个可靠运输协议

在这个编程作业实验中，你将要编写发送和接收运输层的代码，以实现一个简单的可靠数据运输协议。这个实验有两个版本，即比特交替协议版本和 GBN 版本。这个实验应当是有趣的，因为你的实现将与实际情况下所要求的差异很小。

因为可能没有你能够修改其操作系统的独立机器，你的代码将不得不在模拟的硬件/软件环境中执行。然而，为你提供例程的编程接口（即从上层和下层调用你的实体的代码），非常类似于在实际 UNIX 环境中做那些事情的接口。（实际上，在本编程作业中描述的软件接口比起许多教科书中描述的无限循环的发送方和接收方要真实得多。）停止和启动定时器也是模拟的，定时器中断将激活你的定时器处理例程。

这个完整的实验作业以及你所需要的代码，可从本书的 Web 网站 <http://www.pearsonhighered.com/cs-resources> 获得。



Wireshark 实验：探究 TCP

在这个实验中，你将使用 Web 浏览器访问来自某 Web 服务器的一个文件。如同在前面的 Wireshark 实验中一样，你将使用 Wireshark 来俘获到达你计算机的分组。与前面实验不同的是，你也能够从该 Web 服务器下载一个 Wireshark 可读的分组踪迹，记载你从服务器下载文件的过程。在这个服务器踪迹文件里，你将发现自己访问该 Web 服务器所产生的分组。你将分析客户端和服务端踪迹文件，以探究 TCP 的方方面面。特别是你将评估在你的计算机与该 Web 服务器之间 TCP 连接的性能。你将跟踪 TCP 窗口行为、推断分组丢失、重传、流控和拥塞控制行为并估计往返时间。

与所有的 Wireshark 实验一样，该实验的全面描述可在本书 Web 站点 <http://www.pearsonhighered.com/cs-resources> 上找到。



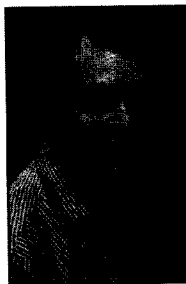
Wireshark 实验：探究 UDP

在这个简短实验中，你将进行分组俘获并分析那些使用 UDP 的你喜爱的应用程序（例如，DNS 或 Skype 这样的多媒体应用）。如我们在 3.3 节中所学的那样，UDP 是一种简单的、不提供不必要服务的运输协议。在这个实验中，你将研究在 UDP 报文段中的各首部字段以及检验和计算。

与所有的 Wireshark 实验一样，该实验的全面描述能够在本书 Web 站点 <http://www.pearsonhighered.com/cs-resources> 上找到。

人物专访

Van Jacobson 就职于谷歌公司，以前是 PARC 的高级研究员。在此之前，他是分组设计（Packet Design）机构的联合创始人和首席科学家。再之前，他是思科公司的首席科学家。在加入思科之前，他是劳伦兹伯克利国家实验室网络研究组的负责人，并在加州大学伯克利分校和斯坦福大学任教。Van 于 2001 年因其在通信网络领域的贡献而获得 ACM SIGCOMM 终身成就奖，于 2002 年因其“对网络拥塞的理解和成功研制用于因特网的拥塞控制机制”而获得 IEEE Kobayashi 奖。他于 2004 年当选为美国国家工程院院士。



Van Jacobson

- 请描述您职业生涯中做过的一两个最令人激动的项目。最大的挑战是什么？